

PROYECTO FINAL

MÓDULO 4

Desarrollo de una herramienta funcional con Low-Code o No-Code

Crea soluciones reales, automatiza procesos y convierte ideas en herramientas que funcionan.

**RÁPIDO**
Desarrolla más en menos tiempo

**FLEXIBLE**
Adapta y mejora sin código

**CONECTADO**
Integra datos, APIs y automatizaciones

**ÚTIL**
Crea valor real para usuarios

**EL OBJETIVO**
Construir una herramienta funcional que resuelva un caso de uso y pueda probarse de forma real.

AUTOMATIZACIÓN
Disparador
↓
Acción
↓
Notificación

BASE DE DATOS

LOWCODE
Dashboard
Usuarios
Solicitudes
Reportes
Configuración

Panel principal
Solicitudes hoy: 24 (+12% vs ayer)
Usuarios activos: 143 (+8% vs ayer)
Tareas pendientes: 7 (-10% vs ayer)
Actividad: [Line chart showing activity over time]
Estado de solicitudes: [Donut chart showing status distribution: Abiertas, En progreso, Cerradas]

FORMULARIO
Nombre: [input]
Email: email@ejemplo.com
Mensaje: [input]
ENVIAR

Mi herramienta
Home Solicitar Mis tareas Priorities Notificaciones



PROYECTO FINAL MÓDULO 4

Desarrollo de una herramienta funcional con Low-Code o No-Code

El ecosistema **low-code** y **no-code** se ha convertido en un componente relevante dentro del desarrollo moderno de soluciones digitales. Estas plataformas permiten acelerar la creación de aplicaciones, reducir la dependencia del código tradicional y facilitar que perfiles no necesariamente técnicos puedan construir herramientas útiles para resolver problemas concretos.

Sin embargo, su valor no está únicamente en la rapidez. Su utilidad principal está en permitir crear soluciones funcionales, validar casos de uso, automatizar procesos y comprobar si una idea tiene sentido antes de invertir en desarrollos más complejos.

A lo largo de este proyecto, el alumno deberá demostrar que comprende cuándo tiene sentido utilizar un enfoque low-code/no-code, qué posibilidades ofrece y cuáles son sus limitaciones.

El objetivo no es crear una aplicación perfecta ni un producto final completamente escalable, sino desarrollar una **herramienta funcional**, de alcance acotado, que resuelva un caso de uso concreto y cuyo flujo principal pueda probarse de forma práctica.

1. Documento ejecutivo

Máximo 2 páginas

El proyecto debe comenzar con un documento ejecutivo de **1 a 2 páginas** que incluya los siguientes apartados:

1.1. Definición del problema o necesidad del usuario

Describir de forma clara el problema que se quiere resolver o la necesidad que se quiere cubrir. También serán válidas aplicaciones que solucionen una necesidad personal.

Debe incluir:

- contexto del caso de uso;
- tipo de usuario o usuarios implicados;
- necesidad principal detectada;
- situación actual que se quiere mejorar.

Ejemplo: una empresa necesita gestionar solicitudes internas, un profesional quiere automatizar la captación de clientes, un equipo quiere registrar incidencias o un usuario necesita consultar información de forma más sencilla.

1.2. Objetivos de la herramienta

Explicar qué pretende resolver la herramienta construida.

Debe quedar claro:

- qué tarea facilita;
- qué proceso mejora;
- qué resultado espera obtener el usuario;
- qué indicadores permitirían valorar si la solución es útil.

Ejemplos de métricas o indicadores:

- reducción de tiempo en una tarea;
- número de solicitudes procesadas;
- mejora en la organización de datos;
- disminución de errores manuales;
- facilidad de uso del flujo principal.

1.3. Justificación del uso de low-code/no-code

Explicar por qué tiene sentido utilizar una herramienta low-code/no-code para este caso.

Se puede justificar por razones como:

- rapidez de desarrollo;
- validación temprana de una idea;
- reducción de costes;
- simplicidad operativa;
- facilidad de iteración;
- posibilidad de construir sin programación avanzada;
- integración sencilla con formularios, bases de datos, automatizaciones o servicios externos.

También se puede explicar por qué no tendría sentido, en esta fase, realizar un desarrollo tradicional completo.

1.4. Alcance y limitaciones

Definir qué funcionalidades incluye la herramienta y cuáles quedan fuera.

Debe quedar claro:

- qué se ha construido;
- qué flujo principal funciona;
- qué funcionalidades no se han incluido;
- qué limitaciones existen por tiempo, complejidad o restricciones de la plataforma utilizada.

El objetivo es que el proyecto sea realista. Es preferible una herramienta sencilla que funcione correctamente a una propuesta demasiado ambiciosa que no pueda probarse.

2. Herramienta funcional

La herramienta debe construirse utilizando al menos una de las plataformas low-code o no-code vistas o documentadas en el módulo, como por ejemplo:

- Manus;
- Genspark;
- Zoer;
- Replit;
- Airtable;
- Bubble;
- Google AI Studio;
- Zapier;
- OpenAI Builder;
- ChatGPT Agent;
- Claude Cowork
- Claude Code
- u otra herramienta similar, siempre que esté justificada.

La herramienta debe mostrar el **flujo principal funcionando** y permitir una interacción real del usuario.

No se valorará únicamente que la herramienta tenga una apariencia visual atractiva o que la idea esté bien explicada. Se valorará especialmente que exista una lógica funcional real.

2.1. Elementos funcionales que se valorarán

Según el tipo de proyecto, se valorarán aspectos como:

- formularios con campos operativos;
- validación básica de campos cuando sea necesario;
- navegación coherente entre pantallas o secciones;
- botones que ejecuten acciones reales;
- login o gestión de usuarios, si el proyecto lo requiere;
- almacenamiento o gestión de datos mediante base de datos, tabla, hoja de cálculo o sistema equivalente;
- automatizaciones simples;
- integración con servicios externos cuando proceda;
- generación de resultados útiles para el usuario;
- conexión coherente entre la parte visual, la lógica de la herramienta y los datos;
- ausencia de errores visibles en el flujo principal.

Ejemplos:

- si hay un formulario, debe poder rellenarse y enviarse;
 - si hay una base de datos, debe tener sentido dentro del flujo;
 - si hay un botón, debe realizar una acción;
 - si hay login, debe estar integrado de forma coherente;
 - si hay una automatización, debe responder a una acción concreta;
 - si hay un pago, reserva, registro o solicitud, el flujo debe poder probarse o explicarse funcionalmente.
-

2.2. Diseño visual

El diseño visual también se tendrá en cuenta, pero será **secundario frente a la funcionalidad**.

Es preferible una herramienta sencilla, clara y funcional que una aplicación visualmente atractiva pero sin lógica real detrás.

El diseño debe ayudar a entender y utilizar la herramienta, pero no sustituye a la funcionalidad.

2.3. Objetivo de la herramienta

El objetivo no es construir un producto final completo ni una aplicación lista para producción, sino una solución funcional de alcance reducido que permita comprobar si el caso de uso planteado tiene sentido.

La herramienta debe demostrar que el alumno ha entendido cómo utilizar el enfoque low-code/no-code para construir una solución práctica.

3. Demostración

Máximo 30 diapositivas

El alumno deberá crear una presentación de máximo **30 diapositivas** que explique el proyecto y demuestre su funcionamiento.

La presentación debe incluir:

- explicación del problema o necesidad;
- justificación de la herramienta elegida;
- descripción del flujo principal;
- recorrido guiado por la solución;
- evidencias de que la herramienta funciona;
- capturas de pantalla, enlaces o demostraciones del proyecto cuando sea posible;
- explicación breve de las limitaciones encontradas.

La demostración debe priorizar la claridad y el enfoque práctico.

No se valorará únicamente la presentación de la idea, sino la evidencia de que la solución funciona tal y como fue planteada.

4. Reflexión individual

Máximo 1 página

El documento debe incluir una reflexión individual de máximo **1 página** sobre los siguientes puntos:

4.1. Aprendizajes obtenidos

Explicar qué se ha aprendido durante el desarrollo del proyecto.

Se puede reflexionar sobre:

- qué significa trabajar con herramientas low-code/no-code;
- qué diferencias hay respecto al desarrollo tradicional;
- cómo se crean flujos funcionales;
- cómo se conectan datos, formularios o automatizaciones;
- qué importancia tiene construir una solución funcional antes de ampliar funcionalidades;
- qué se ha aprendido sobre iterar, probar y corregir errores.

4.2. Limitaciones encontradas

Identificar las principales limitaciones técnicas o prácticas encontradas durante el proyecto.

Por ejemplo:

- restricciones de la plataforma utilizada;
- problemas de personalización;
- dificultades con integraciones;
- limitaciones para escalar la solución;
- errores en formularios, datos, navegación o automatizaciones;
- dificultad para conectar frontend, backend o servicios externos;
- aspectos que no se pudieron implementar por tiempo o complejidad.

4.3. Opinión personal sobre el valor actual de estas herramientas

Valorar de forma crítica si las herramientas low-code/no-code tienen utilidad real.

Se puede comentar:

- si son útiles para aplicaciones internas;
 - si sirven para automatizaciones;
 - si tienen valor para soluciones funcionales rápidas;
 - qué riesgos presentan;
 - qué limitaciones tienen;
 - en qué casos recomendarías su uso;
 - en qué casos sería mejor optar por desarrollo tradicional.
-

5. Criterio principal de valoración

Tal y como se ha indicado en clase, el criterio principal de valoración será la **funcionalidad real de la herramienta**.

En este módulo no se busca únicamente una interfaz visual, una explicación conceptual o una presentación atractiva de la idea. Se busca que el alumno construya una solución que pueda probarse de forma práctica y cuyo flujo principal funcione correctamente.

Se valorará especialmente:

- que el usuario pueda completar el flujo principal;
- que los formularios funcionen;
- que los campos tengan sentido y validación básica cuando proceda;
- que los botones ejecuten acciones reales;
- que exista lógica funcional;
- que los datos se guarden, se consulten o se gestionen de forma coherente;
- que exista conexión entre la interfaz y la lógica de la herramienta;
- que las automatizaciones o integraciones, si existen, estén justificadas y funcionen de forma básica;
- que no haya errores visibles que impidan utilizar la herramienta.

El diseño visual puede mejorar la experiencia de uso, pero no será el criterio principal. Tendrá más valor una herramienta sencilla que funcione correctamente que una propuesta muy estética sin comportamiento real detrás.

6. Entregables

El proyecto final deberá incluir:

1. **Documento ejecutivo** de máximo 2 páginas.
2. **Herramienta funcional** desarrollada con una plataforma low-code/no-code.
3. **Presentación de demostración** de máximo 30 diapositivas.
4. **Reflexión individual** de máximo 1 página.
5. **Acceso a la herramienta funcional**: el alumno deberá entregar un enlace público o privado que permita probar la herramienta. Si la plataforma utilizada no permite compartirla mediante enlace, deberá entregarse un archivo comprimido **.zip** con los archivos necesarios para poder revisarla o ejecutarla localmente, junto con instrucciones básicas de uso.

7. Recomendación final

Antes de entregar, revisa que tu proyecto responde a estas preguntas:

- ¿Qué problema resuelve?
- ¿Quién es el usuario?
- ¿Qué herramienta low-code/no-code he utilizado y por qué?
- ¿Cuál es el flujo principal?
- ¿Puede probarse ese flujo?
- ¿Funcionan los formularios, botones, datos o automatizaciones principales?
- ¿Qué limitaciones tiene?
- ¿Qué he aprendido al construirlo?

El proyecto no tiene que ser perfecto. Debe ser **claro, funcional, justificable y comprobable**.